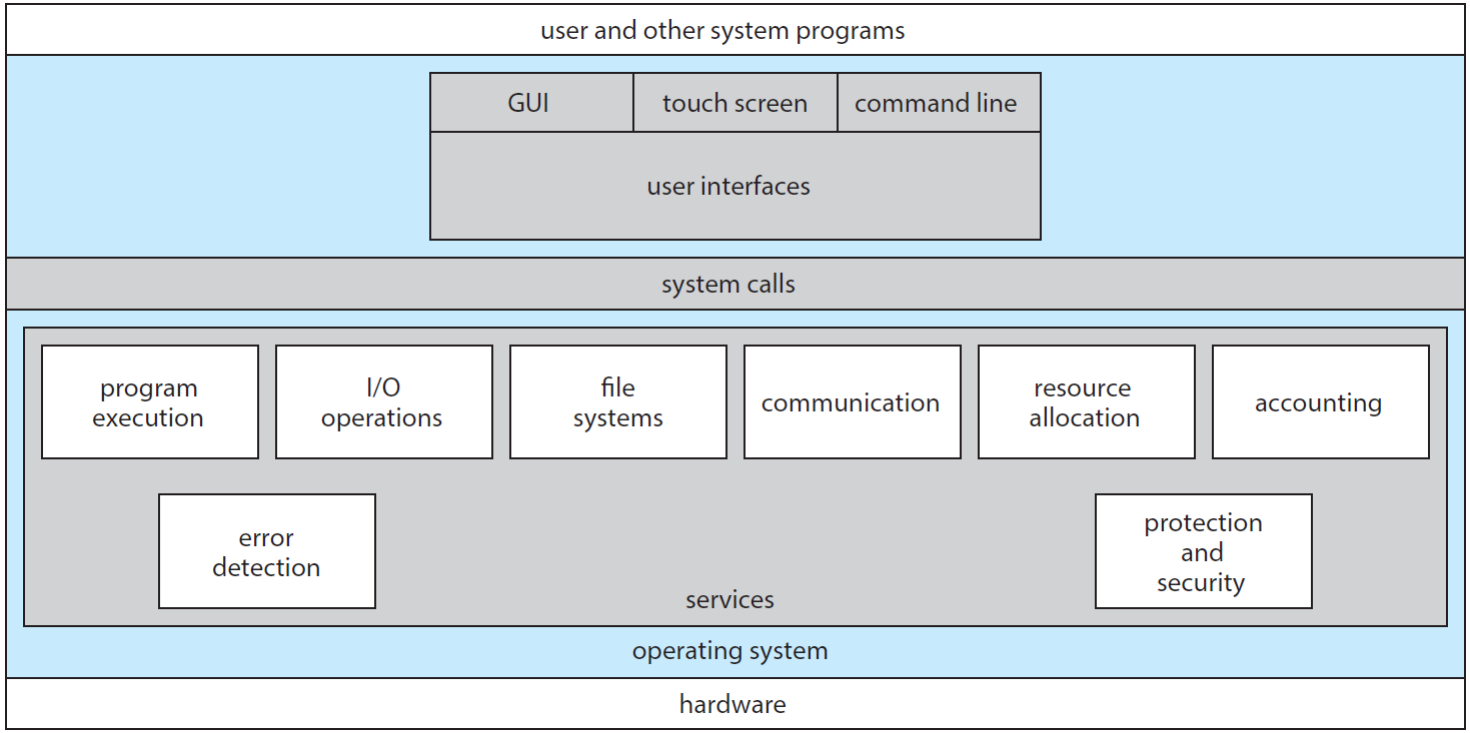
第2章 操作系统结构

- 本章内容
- 1. 功能概述
- 2. 用户接口
- 3. 系统调用
- 4. 系统服务
- 5. 操作系统结构

操作系统提供的功能


user and other system programs
GUI
touch screen
command line
user interfaces
system calls
program execution
I/O operations
file systems
communication
resource allocation
accounting
error detection
protection and security
services
operating system
hardware

操作系统提供的功能

- 用户接口：包括命令行接口、批处理接口及图形用户接口
- 程序执行：将程序装入内存并运行程序
- I/O操作：操作系统必须提供完成I/O操作的手段
- 文件系统操纵：让程序能够读、写、创建和删除文件
- 通信：进程间交换信息

操作系统提供的功能

- 错误检测：通过检测错误确保正确运算
- 资源分配：把资源分配给同时运行的作业
- 统计：记录用户使用资源的类型及数量
- 保护和安全：确保所有对资源的访问是受控的且系统不受外界侵犯

第2章 操作系统结构

- 本章内容
- 1. 功能概述
- 2. 用户接口
- 3. 系统调用
- 4. 系统服务
- 5. 操作系统结构

操作系统的接口

- 操作系统向用户提供了各种使用其服务功能的手段，即提供了操作系统接口
- 操作系统向用户提供的接口
  - 命令接口
  - 图形接口
  - 程序接口

命令解释程序

- 用命令接口进行作业控制的主要方式
  - 脱机控制方式：用户将对作业的控制要求以作业控制说明书的方式提交给系统，由系统按照作业说明书的规定控制作业的执行
  - 联机控制方式：指用户利用系统提供的一组键盘命令或其他操作命令和系统会话，交互式地控制程序的执行

联机命令接口

- 键盘命令可分为：
  - 内部命令：命令功能简单、程序短小、使用频繁。它们在系统初始启动时被引导至内存并常驻内存
  - 外部命令：命令功能较复杂、程序较长、独立作为一个文件驻留在磁盘上，当需要它们时，再从磁盘上调入内存运行
    - PATH环境变量

脱机命令接口

- 脱机命令接口由一组作业控制命令组成
  - 脱机用户不能直接干预作业的运行，他们应事先用作业控制命令写一份作业操作说明书，连同作业一起提交给系统
  - 当系统调度到该作业时，由系统中的命令解释程序对作业说明书上的命令或作业控制语句逐条解释执行

脱机命令接口

- 例

```
files=("dist" "ruoyi-admin.jar" "font" ......)
for file in "${files[@]}"; do
  ......
  if [ -e "$file" ]; then
    echo "check whether $file exists$spaces --------- [✓]"
  else
    ......
  fi
done
......
sudo docker build -f Dockerfile-API -t harbor.xxx.com/wutan/api:latest .
sudo docker build -f Dockerfile-UI -t harbor.xxx.com/wutan/ui:latest .
......
echo "done!"
```

脱机命令接口

- 例

```
@echo off
set rarpath="c:\program files\winrar\"
set logpath=c:\log\
set prefix=anc
rem 获取昨天日期
set YE=%date:~0,4%
set MO=%date:~5,2%
……
set Han=%LY%-%LM%-%LD%
echo 昨天的日期为：%Han%
%logpath:~0,2%
cd %logpath%
%rarpath%rar.exe a %prefix%_server_%Han%.rar server_%Han%.log
```

图形用户接口

- GUI提供基于鼠标的窗口和菜单系统作为接口
  - 通过对出现在屏幕上的对象直接进行操作，以控制和操纵程序的运行
  - 减少或免除了用户的记忆工作量，极大地方便了用户
  - 最为常见的人机接口形式，可以认为图形接口是命令接口的图形化

第2章 操作系统结构

- 本章内容
- 1. 功能概述
- 2. 用户接口
- 3. 系统调用
- 4. 系统服务
- 5. 操作系统结构

系统调用原理

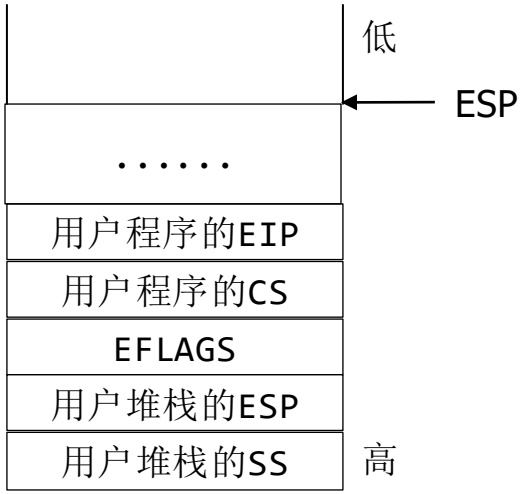
- 系统调用是操作系统向用户程序提供的接口
  - 形式：特殊指令，特定寄存器中存放功能编号
- 参数传递有3种常用方式
  - 寄存器中的参数传递
  - 参数存在内存的一张表（结构体、数组）中，表地址作为寄存器的参数传递
  - 程序把参数压入栈，由操作系统弹出
    - 没那么简单，需讨论

系统调用与栈

- 回顾
  - 系统调用的本质是软中断，使CPU进入内核态
  - 栈和CPU寄存器共同决定了当前程序的执行状态
- 重要常识
  - 应用程序和系统拥有各自的栈：用户栈、内核栈
  - 系统调用时发生栈的切换
- 思考
  - 系统调用参数压栈，哪个栈？
    - 应用程序，当然只能访问用户栈

系统调用与栈

- 继续思考：栈不同，如何衔接参数？
  - 不同的CPU架构，异常/中断时，对栈的处理方法不同
  - x86/x64
    - CPU借助其TSS（任务状态段）找到内核栈，自动把用户栈相关寄存器压入内核栈
    - 但CPU不会自动切换到内核栈
    - 随后处理程序负责切换栈
    - 之后内核可找到用户栈取参数



系统调用与栈

- 继续思考：栈不同，如何衔接参数？
  - 不同的CPU架构，异常/中断时，对栈的处理方法不同
  - RiscV
    - CPU啥也不做，不自动保存，不自动切换栈
    - 处理程序负责保存用户栈相关寄存器
    - 处理程序负责切换栈
    - 内核可根据保存的寄存器数据找到用户栈取参数

系统调用与过程调用的区别

- 运行状态不同
  - 系统调用导致切换到核心态下运行，切换栈
  - 子程序调用不引发任何切换，仍用户态下运行，使用同一个栈
    - 理论上可偷窥调用者栈变量和历史函数的栈遗迹
- 进入方式不同
  - 系统调用通过中断机构进入以改变运行状态
  - 子程序直接[illegible]不涉及运行状态改变

问题2.1：系统调用通过软中断触发，但中断的处理必须简短，否则会耽误接收处理其它中断。如果系统调用需要执行较长的时间，怎么办？

系统调用案例（RiscV）

异常/中断处理程序

```
......
li a0, TRAPFRAME   # 一个结构体
sd sp, 48(a0)
......
ld sp, 8(a0)
# load the address of usertrap()
ld t0, 16(a0)
......
jr t0
```

```
void usertrap(void) {
  ......
  p->trapframe->epc = r_sepc();
  if(r_scause() == 8){  // system call
    ......
    p->trapframe->epc += 4;
    ......
    syscall();
  } else if((which_dev = devintr()) != 0){
    ......
  }
  ......
}
```

用户态程序

```
#define SYS_fork 1
......
li a7, SYS_fork
ecall
......
```

```
uint64 sys_exit(void) {
  ......
}
uint64 sys_fork(void) {
  ......
}
```

```
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
......
};

void syscall(void) {
  ......
  num = p->trapframe->a7;
  ......
  p->trapframe->a0 = syscalls[num]();
  ......
  }
}
```

系统调用的类型

- 系统调用可以大致分为5类
  - 进程控制：如创建、终止进程、等待、唤醒、分配和释放内存
  - 文件管理：如创建文件、删除文件、打开、关闭、读、写
  - 设备管理：如请求设备、释放设备、读、写
  - 信息维护：如读取、设置时间或日期
  - 通信：如创建、删除通信连接，发送、接收消息

第2章 操作系统结构

- 本章内容
- 1. 功能概述
- 2. 用户接口
- 3. 系统调用
- 4. 系统服务
- 5. 操作系统结构

系统服务的类型

- 系统服务是用户角度感受到的各种服务
  - 提供程序开发和执行的便利环境
  - 包括系统调用、程序的装载执行、各种后台服务、各种管理工具、甚至浏览器、游戏

系统服务的类型

- 系统服务可划分为
  - 文件操纵
  - 状态信息
  - 文件修改
  - 程序语言支持
  - 程序调入和执行
  - 通信

API

- 系统调用非常不方便
  - 特定寄存器、特定内存地址，无法用高级语言表达
- Application Programming Interface
  - 操作系统商家提供，封装系统调用，形成C语言函数库，目的是方便应用程序员
  - 思考：API函数运行于内核态还是用户态？
    - 理解：#include，连接，最终成为应用程序的一部分

API

- 现实中，操作系统的设计重点是API
  - 开源系统公开系统调用，但也不鼓励使用，Windows根本不公开系统调用
  - Windows：Win32，Linux/UNIX：POSIX（只是一个公开规范，可理解为交集）
  - 不同版本Windows，系统调用可能有所不同，只要API一样，就可兼容
- Java VM、.Net、C++标准库对API进一步封装

第2章 操作系统结构

- 本章内容
- 1. 功能概述
- 2. 用户接口
- 3. 系统调用
- 4. 系统服务
- 5. 操作系统结构

操作系统的内核结构

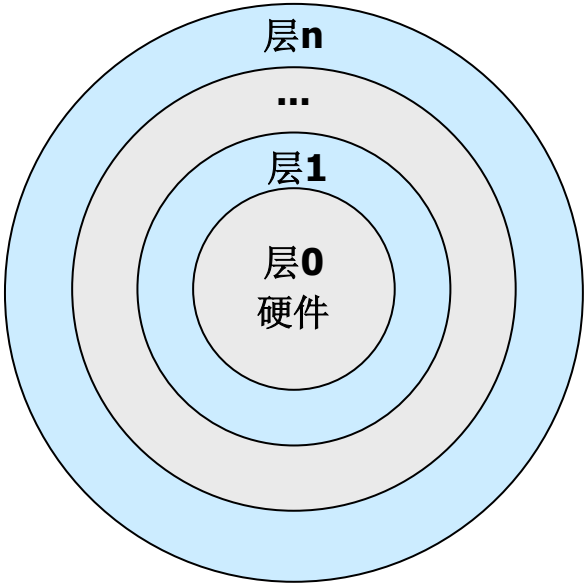
- 操作系统是一个大型系统软件，其内核结构主要有三种
  - 模块结构
  - 层次结构
  - 微内核结构

简单结构

- 一些商业操作系统最初很小、简单且功能有限，没有明确定义的结构
  - MS-DOS，以最小的空间提供最多的功能
    - 不划分模块
    - 其接口和功能层次没有划分清楚
  - 早期UNIX，受硬件功能限制，由2个独立部分组成
    - 系统程序
    - 内核

分层方法

- 操作系统划分为若干层
  - 底层（0层）为硬件，最高层为用户接口
  - 每层只与相邻层次发生直接联系
  - 每层都利用较低层的功能和服务，为较高层隐藏一定的数据结构、操作和硬件的存在



层次结构特点

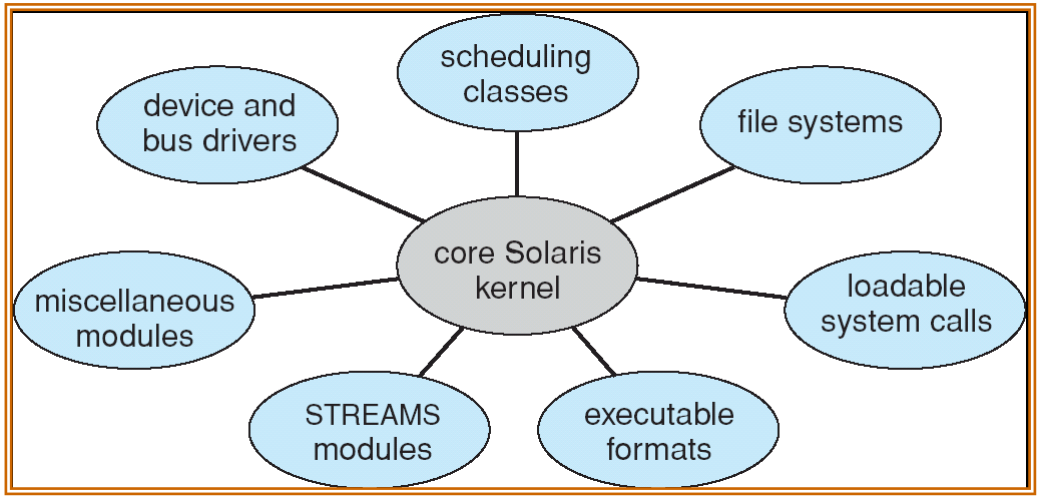
- 层次结构是一种特殊的模块结构
  - 给模块赋予了层次顺序，使调用关系变得有序
  - 在上下两层不变的基础上可以换掉某层，便于移植和扩充
  - 但以牺牲一定的灵活性为代价

模块结构

- 模块结构将操作系统内核按照功能划分为一个个独立的模块，模块之间相对独立，只能通过事先规定好的接口方式来调用
- 特点
  - 效率高
  - 但全局函数使用多造成访问控制困难
  - 结构不清晰，可理解性、可维护性及可移植性差

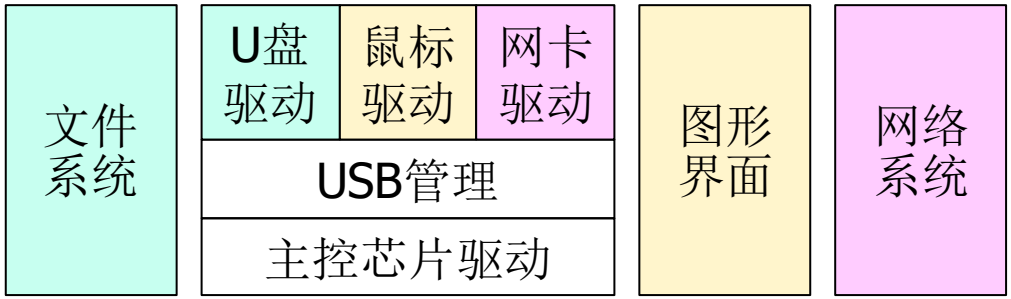
可加载的Solaris模块

- Solaris组织成7个可加载的内核模块及一个核心内核
  - 调度类
  - 文件系统
  - 可执行格式
  - 杂项模块
  - STREAMS模块
  - 可加载的系统调用
  - 设备和总线驱动

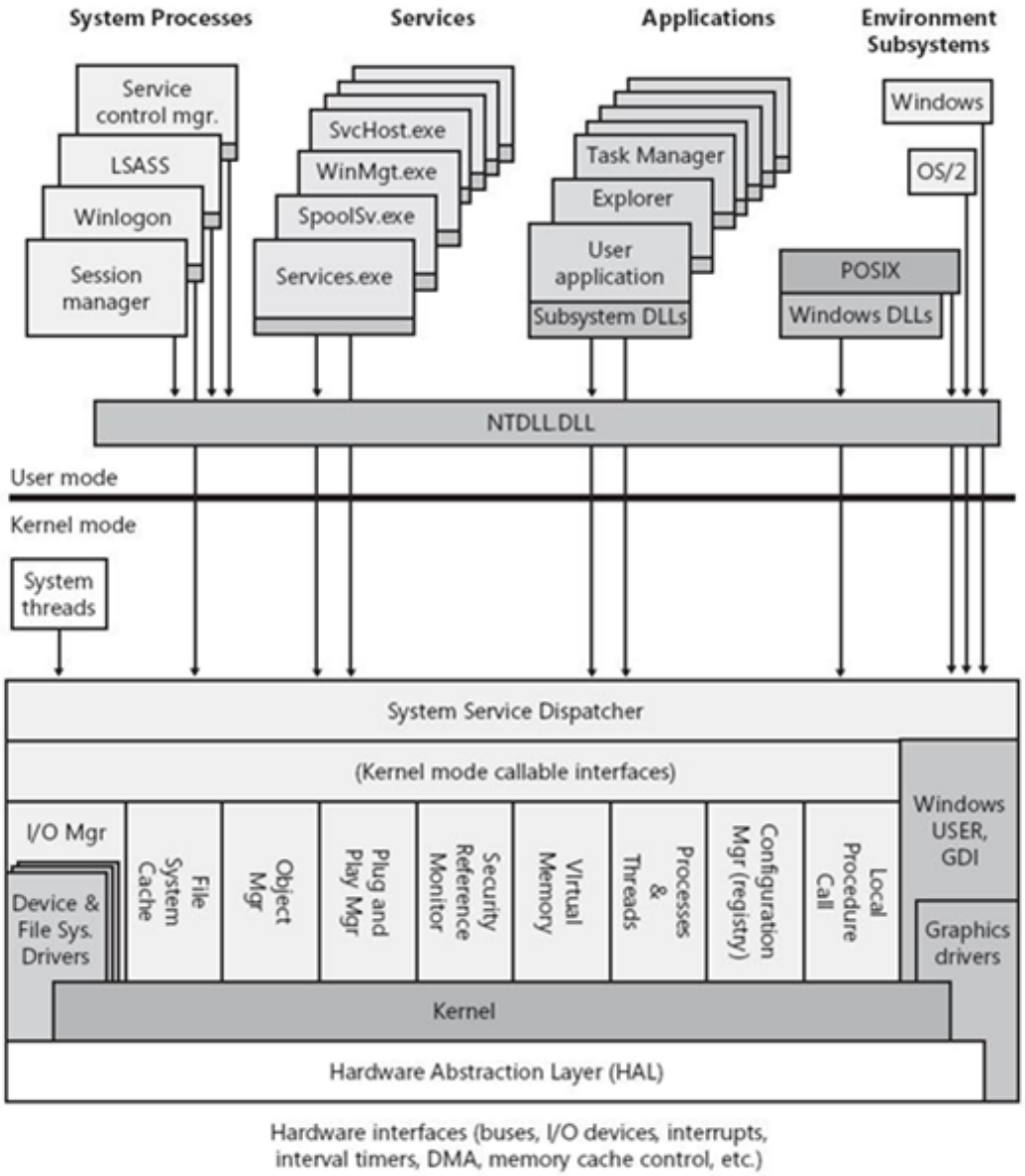


现实的选择

- 有总体的分层
  - 应用程序、API、内核、驱动、硬件
- 内核按功能纵向划分为若干模块
  - 进程、内存、文件、IO
  - 相互依赖、调用
- 每个功能模块内部根据需要进一步分层



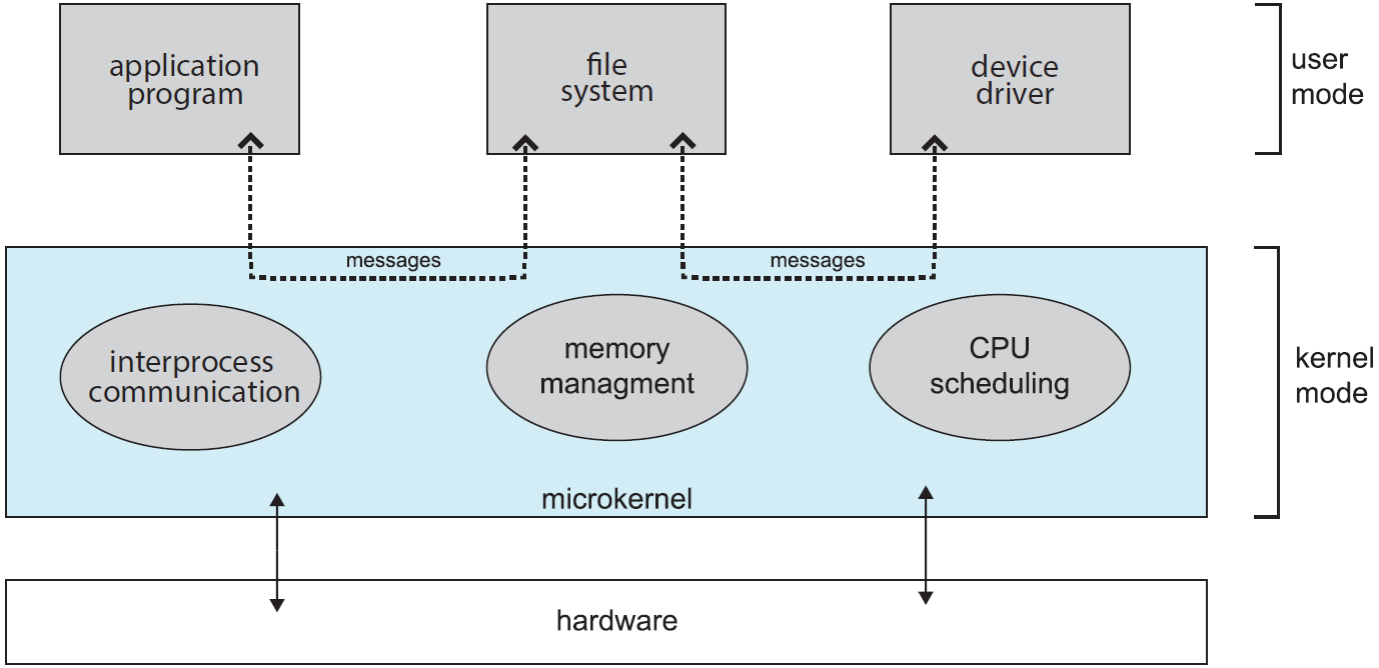
现实的选择


System Processes
Services
Applications
Environment Subsystems
Service control mgr.
LSASS
Winlogon
Session manager
SvcHost.exe
WinMgt.exe
SpoolSv.exe
Services.exe
Task Manager
Explorer
User application
Subsystem DLLs
Windows
OS/2
POSIX
Windows DLLs
NTDLL.DLL
User mode
Kernel mode
System threads
System Service Dispatcher
(Kernel mode callable interfaces)
I/O Mgr
Device & File Sys. Drivers
File System Cache
Object Mgr
Plug and Play Mgr
Security Reference Monitor
Virtual Memory
Processes & Threads
Configuration Mgr (registry)
Local Procedure Call
Windows USER, GDI
Graphics drivers
Kernel
Hardware Abstraction Layer (HAL)
Hardware interfaces (buses, I/O devices, interrupts, interval timers, DMA, memory cache control, etc.)

微内核

执行和决策，谁留在内核中？

- 基本思想：执行与决策分离，缩小内核规模
  - 将所有非基本部分从内核中移走，并将它们实现为系统程序或用户程序
  - 微内核包括最小的进程和内存管理及通信功能



微内核

- 优点
  - 降低了开发难度，容易扩充操作系统
  - 容易从一种硬件平台移植到另一种硬件平台
  - 更安全可靠
- 不足
  - 增加大量系统调用和进程切换，总开销增加导致系统性能下降

微内核

- 现实情况：太多系统调用和进程切换，效率低下无法广泛应用
  - 大多数主流系统属于宏内核（也称单内核）
  - Mach：卡内基梅隆大学开发，真正的微内核，性能问题严重，失败
  - Windows，少数对性能不敏感的功能被剥离
    - winlogon.exe，lsass.exe，smss.exe与登录会话有关
    - csrss.exe与DOS或POSIX子系统有关
  - 鸿蒙，微内核，采用大量优化解决效率问题

练习题

- 系统调用的目的是什么？
- 从应用程序的视角来看，异常和中断的区别是什么？
- 在发生CPU的特权级切换时，CPU会自动保存当前的执行状态，包括程序计器(PC)、栈指针(Stack Pointer, SP)等。请分析：如果不保存PC和SP，会出现什么问题？

练习题

- 系统调用和库函数或API之间是什么关系？
- 操作系统提供的系统调用有哪几种参数传递的方法？
- 请解释说明系统调用机制涉及的概念：访管指令、系统调用号、参数传递、系统调用表、系统调用实现函数。

选择题1

- 操作系统提供给编程人员的接口是_____。

A.系统调用　　B.子程序

C.库函数　　D.高级语言

考研题

- 下列选项中，操作系统提供给应用程序的接口是____。10

A. 系统调用　　B. 中断

C. 库函数　　D. 原语

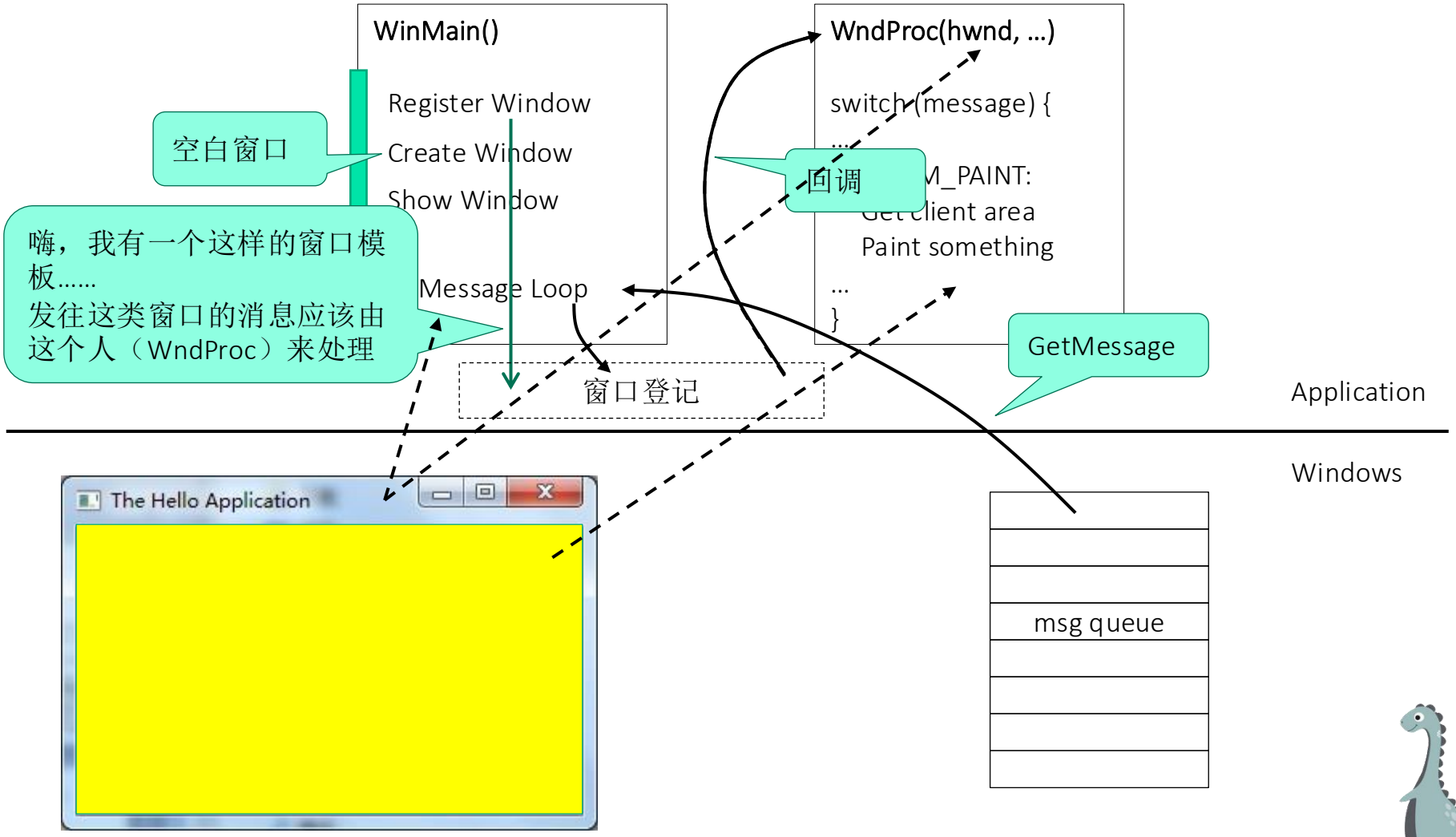
API案例：最简Win32程序

- 重要基础概念
  - 消息与消息队列
    - 系统发送的一种结构，用于向应用程序通知某些事件
      - 窗口handle，指示消息发送到哪个窗口
      - 消息ID，指示这是哪种类型的消息
      - wParam和lParam，两个32位参数，不同消息有不同的用途
    - 每个UI线程都有一个消息队列（由系统维护），其程序可以通过API函数GetMessage()获取消息
    - Windows应用程序启动后不会立即退出，其整个生命周期中的主要任务是处理消息
    - 为了响应不时产生的消息，应用程序必须在程序中包含一个“消息循环”

API案例：最简Win32程序

- 重要基础概念
  - 回调函数
    - 一个由应用程序员编写的函数，其指针（地址）被传递给系统，并由系统调用
  - 句柄（Handle）
    - 基本上，是个整数，用于标识某个系统对象的ID号
    - 系统对象：属于系统内核但可由应用程序操作的对象，如线程、窗口、菜单、按钮、屏幕区域等
    - 出于安全考虑，内核不能将系统对象的指针（地址）传递给应用程序，而是传递一个ID
    - 应用程序调用API（以句柄为参数）来操作系统对象

API案例：最简Win32程序



API案例：最简Win32程序

```
#include <windows.h>
LONG WINAPI WndProc (HWND, UINT, WPARAM, LPARAM);

int WINAPI WinMain (HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpszCmdLine, int nCmdShow)
{
    WNDCLASS wc;
    HWND hwnd;
    MSG msg;

    wc.style = 0;                                          // Class style
    wc.lpfnWndProc = (WNDPROC) WndProc;                    // Window procedure
    wc.cbClsExtra = 0;                                     // Class extra bytes
    wc.cbWndExtra = 0;                                     // Window extra bytes
    wc.hInstance = hInstance;                              // Instance handle
    wc.hIcon = LoadIcon (NULL, IDI_WINLOGO);               // Icon handle
    wc.hCursor = LoadCursor (NULL, IDC_ARROW);             // Cursor handle
    wc.hbrBackground = (HBRUSH) (COLOR_WINDOW + 1);        // Background color
    wc.lpszMenuName = NULL;                                // Menu name
    wc.lpszClassName = "MyWndClass";                       // WNDCLASS name

    RegisterClass (&wc);
    ……
```

为窗口模板设置一些样式

API案例：最简Win32程序

```
    ……
    hwnd = CreateWindow (
        "MyWndClass",                   // WNDCLASS name
        "SDK Application",              // Window title
        WS_OVERLAPPEDWINDOW,            // Window style
        CW_USEDEFAULT,                  // Horizontal position
        CW_USEDEFAULT,                  // Vertical position
        CW_USEDEFAULT,                  // Initial width
        CW_USEDEFAULT,                  // Initial height
        HWND_DESKTOP,                   // Handle of parent window
        NULL,                           // Menu handle
        hInstance,                      // Application's instance handle
        NULL                            // Window-creation data
    );
    ShowWindow (hwnd, nCmdShow);
    UpdateWindow (hwnd);

    while (GetMessage (&msg, NULL, 0, 0)) {
        TranslateMessage (&msg);
        DispatchMessage (&msg);
    }
    return msg.wParam;
}
```

为窗口设置一些样式

问题2.2：消息循环无限执行，为何CPU占用率没有提高？

API案例：最简Win32程序

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam,
    LPARAM lParam)
{
    PAINTSTRUCT ps;
    HDC hdc;

    switch (message) {

    case WM_PAINT:
        hdc = BeginPaint (hwnd, &ps);
        Ellipse (hdc, 0, 0, 200, 100);
        EndPaint (hwnd, &ps);
        return 0;

    case WM_DESTROY:
        PostQuitMessage (0);
        return 0;
    }
    return DefWindowProc (hwnd, message, wParam, lParam);
}
```